

ROS2 Imitation Learning Pipeline — Project Report

A hardware-agnostic ROS2 pipeline for teaching a robot arm to perform manipulation tasks through imitation learning. Built in eight stages, it covers the full loop from teleoperation and episode recording through policy training and autonomous inference, integrated with the MyBotShop Robotic Webserver platform.

Author: Touseef Qamar

Repository: <https://github.com/TouseefQ/ros2-il-pipeline>

Table of Contents

1. [Executive Summary](#)
 2. [Project Purpose and Goals](#)
 3. [System Architecture](#)
 4. [Development Methodology](#)
 5. [Stage 1 — Foundation](#)
 6. [Stage 2 — Data Collection](#)
 7. [Stage 3 — Webserver Bridge](#)
 8. [Stage 4 — Dataset Tooling](#)
 9. [Stage 5 — Behavioural Cloning Training](#)
 10. [Stage 6 — Inference Node](#)
 11. [Stage 7 — ACT Training](#)
 12. [Stage 8 — Integration and Demo](#)
 13. [Challenges and How They Were Solved](#)
 14. [Results and Outcomes](#)
 15. [Future Work and Extensions](#)
 16. [Appendix](#)
-

1. Executive Summary

- A complete, hardware-agnostic imitation learning pipeline built on ROS2 Humble.
- Covers the full loop: teleoperation → episode recording → policy training → autonomous inference.
- A human operator demonstrates a task; each demonstration is recorded as an HDF5 episode.

- Recorded episodes are used to train a policy via Behavioural Cloning (BC) or ACT.
 - The trained policy is loaded by a ROS2 inference node that drives the arm autonomously at runtime.
 - The entire stack runs in Docker and is validated by a GitHub Actions CI pipeline.
-

2. Project Purpose and Goals

- Build the infrastructure that makes imitation learning practical on a real robot.
 - Core capabilities: reliable episode recording, correct dataset handling, portable model checkpoints, and safe real-time inference.
 - Designed around the MyBotShop Robotic Webserver as the intended hardware platform, but operates fully without it.
 - A MoveIt2 Panda simulation on fake hardware was used throughout development.
 - Robot identity is a configuration parameter — deploying to different hardware requires only a `pipeline.yaml` update.
 - Out of scope: physical robot deployment, temporal ensemble, diffusion policy, and visual policies (architecture accommodates them).
-

3. System Architecture

- Five ROS2 packages: `il_interfaces`, `il_pipeline_bringup`, `il_data_collection`, `il_webserver_bridge`, `il_inference`.
 - Standalone `training/` directory for offline learning; `tools/` directory for dataset utilities.
 - Sensor data flows in via `/joint_states`, `/gripper_state`, and optional camera topics.
 - `data_collector_node` writes observations to HDF5 episodes; training scripts consume them and produce a `.pt` checkpoint.
 - `il_inference_node` loads the checkpoint and publishes joint trajectory commands back to the robot.
 - The webserver bridge forwards IK solutions from the MyBotShop WebSocket feed during teleoperation and hosts an HTTP control panel.
 - Technology stack: ROS2 Humble, MoveIt2 (Franka Panda on fake hardware), HDF5, PyTorch.
-

4. Development Methodology

- Built in eight sequential stages: Foundation → Data Collection → Webserver Bridge → Dataset Tooling → BC Training → Inference Node → ACT Training → Integration.
 - Each stage was committed independently after verification, so every component could be tested in isolation.
 - Hardware-agnosticism was a first-class constraint: all robot-specific values live in `pipeline.yaml`.
 - The pipeline adapts to a different arm without code changes — only config updates needed.
 - Development ran entirely inside Docker from the start.
 - Separate Docker images for the ROS2 runtime and the PyTorch training environment ensure reproducibility on any machine.
-

5. Stage 1 — Foundation

- Established the complete project scaffold: all five ROS2 packages with `package.xml`, `setup.py`, and resource markers.
 - Custom ROS2 interface definitions in `il_interfaces`: messages, services, and the `RunEpisode` action.
 - Four launch files and the central `pipeline.yaml` configuration.
 - Docker setup with separate runtime and training images.
 - Every node created as a skeleton with parameter declarations, topic/service wiring, and `# TODO Stage N`: markers — no logic implemented yet.
 - `il_interfaces` was treated as the system contract: defined first to lock in data shapes before any implementation began.
-

6. Stage 2 — Data Collection

- `DataCollectorNode` subscribes to `/joint_states`, `/gripper_state`, and optional camera topics.
- Uses a TF2 listener to look up end-effector pose at each tick.
- Reorders incoming joint data to match the configured `joint_names` order.
- Exposes `start_recording` and `stop_recording` services that create and finalise `EpisodeWriter` instances.
- Each episode is saved as a single HDF5 file with a `meta/` group (episode ID, collection mode, timestamps, step count).

- Observation arrays: `joint_pos`, `joint_vel`, `eef_pos`, `eef_quat`, `gripper_state`, and optionally gzip-compressed camera images.
 - Action arrays: `joint_pos_delta` (for BC) and `joint_pos_abs` (for ACT).
 - `DatasetManagerNode` scans the episodes directory on a timer and publishes a JSON summary to `/il/dataset_stats`.
-

7. Stage 3 — Webserver Bridge

- Integrated the MyBotShop Robotic Webserver as a pure client — no webserver source files were modified.
 - `TeleopBridgeNode` connects as a WebSocket client to the IK solver broadcast endpoint on port 9001.
 - Each joint solution received is reordered to match configured joint names and forwarded to `/joint_commands`.
 - Finger position broadcasts are normalised and forwarded to `/gripper_command`.
 - The webserver could not be used during development: it is closed-source with a hard dependency on `robot_arm_ik`, a proprietary node that requires physical hardware.
 - A `panda_ik_ws_node` was written as a stand-in, broadcasting MoveIt2-planned joint positions on the same port.
 - `WebserverActionNode` hosts a minimal HTTP control panel on port 9010, using a thread-safe queue to bridge HTTP requests to ROS2 service calls.
-

8. Stage 4 — Dataset Tooling

- Three standalone Python tools added for inspecting and exporting the collected dataset.
 - `dataset_stats.py`: scans a directory of HDF5 episodes; reports episode count, step counts, average duration, and collection mode breakdown; flags corrupt files.
 - `dataset_viewer.py`: opens a single episode and renders a 6-panel matplotlib figure (joint positions, EEf XY path, EEf Z height, gripper state, action trajectories); with `--show_images` it animates camera frames alongside a live gripper cursor.
 - `export_to_lerobot.py`: converts the full dataset to HuggingFace LeRobot v2 Parquet format, writing per-episode Parquet files plus `meta/info.json`, `meta/episodes.jsonl`, and `meta/stats.json` with per-feature normalisation statistics.
-

9. Stage 5 — Behavioural Cloning Training

- `DemonstrationDataset` loads all episode HDF5 data into memory at construction — no file I/O in the training hot loop.
 - Uses an episode-level (not step-level) train/validation split to prevent data leakage.
 - `Normalizer` fits per-feature z-score statistics on training episodes only and is serialised into every checkpoint.
 - The inference node can denormalise predictions without access to the original training data.
 - `MLPPolicy`: concatenates joint positions, velocities, EEJ position, and quaternion into a fully connected network predicting joint position deltas.
 - `CNNMLPPolicy`: adds a frozen pretrained ResNet-18 backbone per camera (512-dim visual features) fused with the state embedding.
 - Training loop (`train_bc.py`): AdamW with cosine annealing; saves `best.pt`, periodic epoch checkpoints, and `final.pt`.
-

10. Stage 6 — Inference Node

- `ILInferenceNode` loads a checkpoint at startup and runs at a configurable `control_hz`.
 - At each tick: looks up end-effector pose via TF2, assembles an observation dictionary, runs the policy, clamps output to joint limits with `np.clip`, and publishes a `JointTrajectory` command.
 - Autonomous control toggled via the `/il/autonomous_mode` Bool topic.
 - `/il/emergency_stop` provides a latching halt that blocks re-activation until explicitly cleared.
 - `RunEpisode` action server allows bounded episodes to be triggered programmatically, streaming per-step feedback and respecting cancellation and emergency stop mid-run.
 - `PolicyLoader` adds `training_src_dir` to `sys.path` at load time — training model classes are importable at runtime without duplicating code into the ROS2 package.
-

11. Stage 7 — ACT Training

- ACT (Action Chunking with Transformers) addresses BC's tendency to accumulate errors by predicting a chunk of future joint positions per forward pass.
- **CVAE encoder**: takes current state and a ground-truth action chunk, adds a learnable CLS token and sinusoidal positional encoding, and produces a latent distribution via a Transformer encoder.

- **Transformer decoder:** uses state and sampled latent as memory; learnable query embeddings produce the full predicted action chunk.
 - At inference the latent is set to zero (prior mean), bypassing the encoder entirely.
 - Training loop (`train_act.py`): combined loss of L1 action reconstruction + KL divergence weighted by `kl_weight` (default 10.0); separate TensorBoard scalars per term.
 - `ACTPolicyLoader` returns only the first step of the predicted chunk per call, re-predicting at each control tick.
-

12. Stage 8 — Integration and Demo

- Entire stack tied together into a single launch command.
 - Rewritten `full_pipeline.launch.py`: brings up MoveIt2 simulator, data collection nodes, webserver bridge, and inference node in one command.
 - Accepts `checkpoint`, `algorithm`, and `training_src_dir` as launch arguments — policy can be swapped without editing config files.
 - End-to-end workflow verified in Docker from a clean clone: collect episodes → train BC → restart stack → enable autonomous mode → arm moves in RViz2.
 - build CI job: runs `colcon build` inside `ros:humble` and verifies all five packages appear in `ros2 pkg list`.
 - lint CI job: runs `flake8` over all Python source in `ros2_ws/src/`, `training/`, and `tools/`.
-

13. Challenges and How They Were Solved

1. **No module named 'datasets' at inference startup** — inference node could not import training model classes at runtime. Solved by introducing the `training_src_dir` parameter that adds the `training/` directory to `sys.path` at checkpoint load time.
2. **ROS2 parameter override precedence bug** — launch files were passing `training_src_dir` as an empty string, silently overriding the value set in `pipeline.yaml`. Fixed by removing the redundant argument declaration so the YAML value takes effect.
3. **Jerky arm motion during inference** — `JointTrajectory.time_from_start` was set too low, causing the controller to rush between waypoints. Increasing it to 300 ms produced smooth motion.
4. **docker exec without ROS sourced** — ad-hoc `docker exec` commands failed because neither the ROS2 nor workspace overlays were sourced. Resolved by prefixing all `exec` commands with the two `source` calls.

5. **WebSocket IK broadcast unavailable in simulation** — the MyBotShop webserver was not accessible during development. Solved by writing `panda_ik_ws_node` as a drop-in stand-in that broadcasts MoveIt2-planned joint positions on the same port.
-

14. Results and Outcomes

- The full collection-to-autonomous-control loop was demonstrated end-to-end in Docker.
- Episodes collected via the HTTP control panel; BC policy trained in the training container; arm moving autonomously in RViz2.
- Policy quality scales with demonstration count:

Episodes	Expected result
< 10	Near-random motion
20–50	Rough approximation of demonstrated behaviour
100+	Consistent task execution

- Known limitations:
 - Pipeline has not been tested on a physical robot (no live MyBotShop webserver access was available).
 - Visual policies are implemented but untested with real camera streams.
 - ACT re-predicts at every step — temporal ensemble is not yet implemented.
 - `panda_ik_ws_node` broadcasts only planned waypoints, not true IK driven by a human operator.
-

15. Future Work and Extensions

- **Physical robot connection:** once live webserver access is available, `TeleopBridgeNode` connects without code changes and `panda_ik_ws_node` is simply not launched.
- **Temporal ensemble for ACT:** execute a predicted chunk before re-predicting rather than discarding all but the first step — reduces jitter and is a straightforward extension to `ACTPolicyLoader`.
- **Visual policies with real cameras:** already supported by the data collection and training stack; requires only live camera feeds to activate.
- **Diffusion Policy:** a third algorithm option; the `PolicyLoader` dispatch pattern accommodates it cleanly.
- **Multi-task or language-conditioned policies:** feasible once a sufficiently diverse demonstration dataset is available.

16. Appendix

A. ROS2 Interface Reference

Topic / Service / Action	Type	Direction
/joint_states	sensor_msgs/ JointState	in
/gripper_state	std_msgs/Float32	in
/camera/wrist/ image_raw	sensor_msgs/Image	in (optional)
/camera/top/image_raw	sensor_msgs/Image	in (optional)
/joint_commands	trajectory_msgs/ JointTrajectory	out
/gripper_command	std_msgs/Float32	out
/il/observation_frame	il_interfaces/ ObservationFrame	pub
/il/episode_status	std_msgs/String	pub
/il/dataset_stats	std_msgs/String (JSON)	pub
/il/pipeline_control	std_msgs/String (JSON)	pub
/il/autonomous_mode	std_msgs/Bool	in
/il/emergency_stop	std_msgs/Bool	in
/data_collector/ start_recording	il_interfaces/srv/ StartRecording	srv
/data_collector/ stop_recording	il_interfaces/srv/ StopRecording	srv
/il_inference/ load_policy	il_interfaces/srv/ LoadPolicy	srv
/il_inference/ run_episode	il_interfaces/ action/RunEpisode	action

B. pipeline.yaml Configuration Reference

Parameter	Default	Description
robot.name	panda	Robot name
robot.num_joints	7	Number of arm joints
robot.joint_names	[panda_joint1 ... 7]	Ordered joint names
robot.base_frame	panda_link0	TF2 root frame
robot.eef_link	panda_hand	End-effector TF2 frame
data_collection.outpu	/workspace/data/	HDF5 episode output path

Parameter	Default	Description
t_dir	episodes	
data_collection.recording_hz	10.0	Observation sampling rate
data_collection.record_images	false	Enable camera recording
inference.default_checkpoint	/workspace/data/models/best.pt	Checkpoint loaded at startup
inference.training_src_dir	/workspace/training	Added to sys.path for model imports
inference.control_hz	10.0	Policy inference rate
inference.action_chunk_size	10	ACT chunk length
webserver.host	localhost	MyBotShop webserver host
webserver.port	9000	Webserver UI port
webserver.ik_ws_port	9001	IK WebSocket port
webserver.action_server_port	9010	HTTP control panel port

C. HDF5 Episode Schema

```

ep_NNN.hdf5
├── meta/
│   ├── episode_id      str
│   ├── robot_description str
│   ├── collection_mode  str      "teleop" | "kinesthetic" | "scripted"
│   ├── timestamp_start  float64
│   ├── timestamp_end    float64
│   ├── num_steps        int
│   └── record_images    bool
└── data/
    ├── observation/
    │   ├── joint_pos      (T, N)  float32
    │   ├── joint_vel      (T, N)  float32
    │   ├── eef_pos        (T, 3)  float32  - xyz
    │   ├── eef_quat       (T, 4)  float32  - xyzw
    │   ├── gripper_state  (T, 1)  float32  - 0.0-1.0
    │   └── images/
    │       ├── camera_wrist (T, H, W, 3) uint8  optional
    │       └── camera_top   (T, H, W, 3) uint8  optional
    └── action/
        ├── joint_pos_delta (T, N)  float32  - used by BC
        └── joint_pos_abs   (T, N)  float32  - used by ACT

```

D. Docker Command Reference

```

# Build all images
docker compose -f docker/docker-compose.yml build

# Start runtime stack (simulator + all nodes)
docker compose -f docker/docker-compose.yml up

```

```

# Train BC policy (CPU)
docker compose -f docker/docker-compose.yml --profile training run --rm training \
    python training/train_bc.py --config \
/workspace/training/configs/bc_config.yaml

# Train ACT policy (CPU)
docker compose -f docker/docker-compose.yml --profile training run --rm training \
    python training/train_act.py --config \
/workspace/training/configs/act_config.yaml

# Enable autonomous mode
docker exec ros2-il-pipeline-ros-1 bash -c \
    "source /opt/ros/humble/setup.bash && source \
/workspace/ros2_ws/install/setup.bash && \
    ros2 topic pub --once /il/autonomous_mode std_msgs/Bool 'data: true'"

# Emergency stop
docker exec ros2-il-pipeline-ros-1 bash -c \
    "source /opt/ros/humble/setup.bash && source \
/workspace/ros2_ws/install/setup.bash && \
    ros2 topic pub --once /il/emergency_stop std_msgs/Bool 'data: true'"

```

E. Glossary

Term	Definition
IL	Imitation Learning — learning a policy from human demonstrations
BC	Behavioural Cloning — supervised regression from observations to actions
ACT	Action Chunking with Transformers — predicts a sequence of future actions per forward pass
CVAE	Conditional Variational Autoencoder — used in ACT to encode the action chunk into a latent distribution
EEF	End-Effector — the tip of the robot arm (gripper mount)
HDF5	Hierarchical Data Format v5 — binary file format used for episode storage
chunk size	Number of future timesteps ACT predicts in a single forward pass
KL weight (β)	Scalar balancing reconstruction loss vs KL divergence in ACT training
training_src_dir	Path added to <code>sys.path</code> at inference time so model classes are importable from the training directory
temporal ensemble	ACT inference strategy where a full predicted chunk is executed before re-predicting (not yet implemented)

